

Intelligenza Artificiale e Machine Learning preparazione all'Esonero

Mattia Cosimi

November 2024

1 Problem Solving e Search

1.1 Introduzione

Il processo di ricerca comporta i seguenti passi: scegliere (tra le foglie dell'albero) un nodo da "espandere", secondo una strategia. Controllare se il nodo scelto è un obiettivo, se non lo è, "espandere" il nodo: generare i suoi figli, ciascuno dei quali contiene uno stato risultante dall'applicazione di un operatore allo stato del nodo espanso. La collezione di nodi in attesa di essere espansi è chiamata **frontiera**. La frontiera è implementata da una struttura che chiamiamo "coda" (queue).

1.2 Tree Search

```
function TREE-SEARCH(problema, strategia) returns una soluzione o
                                     un fallimento

  Inizializzare l'albero di ricerca usando lo stato iniziale del
  problema.

  loop do
    • Se non ci sono candidati per l'espansione (frontiera vuota)
      riportare fallimento.
    • Scegliere una foglia per l'espansione (in base alla
      strategia).
    • Se il nodo scelto contiene uno stato obiettivo, riportare la
      soluzione corrispondente (costruita seguendo i "puntatori al
      padre").
    • Altrimenti espandere il nodo e aggiungere i nodi risultanti
      all'albero di ricerca.
  end
```

```
function TREE-SEARCH(problem) returns a solution or failure

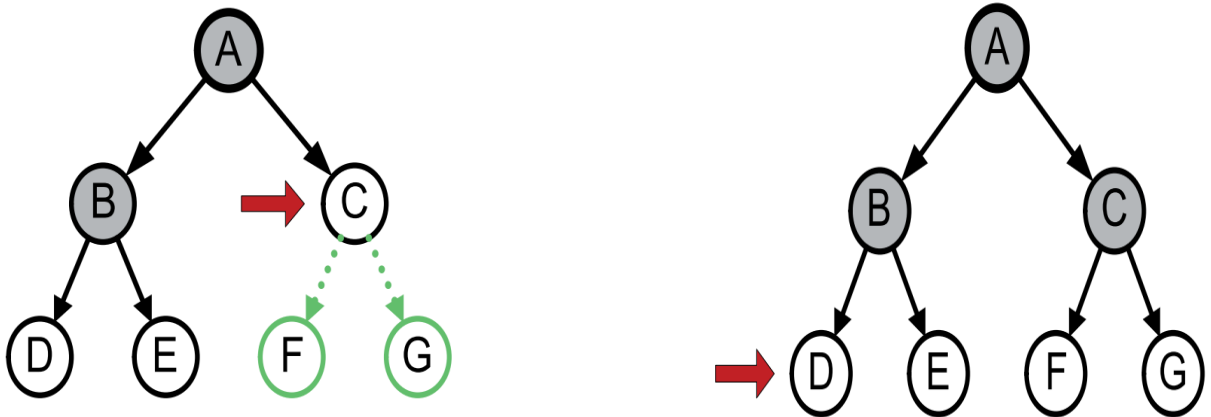
  fringe ← MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))
  loop do
    If EMPTY?(fringe) then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return SOLUTION(node)
    fringe ← QUEUING-FN(fringe, EXPAND(node, OPERATORS(problem)))
  end
```

2 Ricerca Cieca

2.1 Ricerca in Ampiezza (Breadth-First-Search)

Si espande il nodo radice, si espandono i nodi generati dalla radice, si espandono i loro successori e così via.

```
function BREADTH-FIRST-SEARCH(problem) returns a solution or failure
    fringe ← MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))
    loop do
        If EMPTY?(fringe) then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return SOLUTION(node)
        fringe ← ENQUEUE-AT-END(fringe, EXPAND(node, OPERATORS(problem)))
    end
```



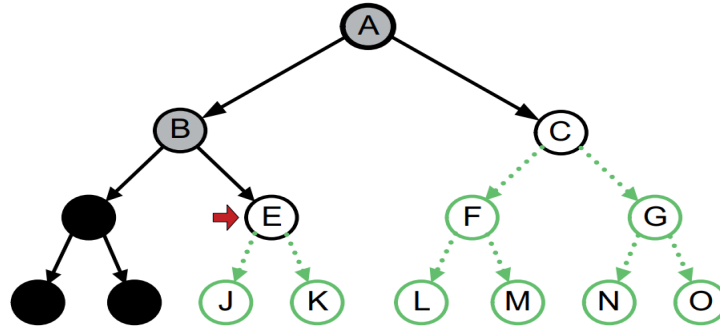
Trova sicuramente la soluzione realizzabile con il cammino più breve, ma il cammino più breve non è necessariamente quello a costo minimo. È **completo**. È **ottimale** se il costo del cammino è una funzione monotona non decrescente della profondità del nodo. La **Complessità in tempo e in spazio**: $O(b^d)$ dove b è il fattore di ramificazione dell'albero (ogni nodo ha al massimo b figli) e d è la lunghezza minima di un cammino dal nodo iniziale alla soluzione.

2.2 Ricerca guidata dal costo

Modifica la ricerca in ampiezza espandendo il nodo nella frontiera con costo più basso. $g(n)$ è il costo dalla radice a n . È completo e ottimale se il costo di ogni step ($g(\text{successor}(n)) - g(n)$) è sempre maggiore o uguale a una costante positiva.

2.3 Ricerca in profondità (Depth-First-Search)

Nella ricerca in profondità si procede come segue: si espande il nodo radice, si procede espandendo sempre per primo il nodo più profondo nella frontiera corrente dell'albero di ricerca.



```
function DEPTH-FIRST-SEARCH(problem) returns a solution or failure
    fringe ← MAKE-QUEUE(MAKE-NODE( INITIAL-STATE[problem]))
    loop do
        If EMPTY?(fringe) then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        fringe ← ENQUEUE-AT-FRONT(fringe, EXPAND(node, OPERATORS(problem)))
    end
```

Figure 1: Depth-First Search

Quando si espande un nodo non obiettivo e senza figli, si fa backtracking ovvero si torna indietro fino all'ultimo nodo in cui è possibile effettuare una scelta. Non è né **completa** né **ottimale**. Complessità in tempo $O(b^m)$ dove m è la profondità massima dell'albero di ricerca. Complessità in spazio $O(b \cdot m)$

2.4 Ricerca in profondità limitata

Opera come la ricerca in profondità imponendo, però, un limite alla profondità massima dei cammini: un nodo viene espanso solo se la lunghezza del cammino corrispondente è minore del massimo stabilito. Se non viene trovata alcuna soluzione l'algoritmo restituisce il valore speciale taglio se alcuni nodi non sono stati espansi per il limite della profondità, altrimenti fallimento. È **completa** se il problema è tale che, se ha soluzione, allora esiste una soluzione di lunghezza minore o uguale al limite massimo. Non è **ottimale**. Complessità in tempo $O(b^l)$ dove l è il limite di profondità fissato. Complessità in spazio $O(b \cdot l)$

2.5 Ricerca per approfondimenti successivi (Iterative-Deepening Search)

Non sempre si conosce un limite adeguato per la ricerca in profondità limitata. Questa strategia evita il problema della scelta di un limite adeguato provando iterativamente tutti i limiti possibili.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns solution
                                         or failure
for depth = 0 to  $\infty$  do
    If DEPTH-LIMITED-SEARCH(problem, depth) succeeds
    then return its result
end
```

Figure 2: Iterative-Deepening Search

È **completa e ottimale** (sotto le stesse ipotesi della ricerca in ampiezza). Complessità in spazio $O(b \cdot d)$ dove d è la profondità minima di una soluzione. Complessità in tempo $O(b^d)$.

2.6 Graph Search

Nasce dall'esigenza di evitare la ripetizione degli stati. Per questo al tree search viene aggiunta una lista chiusa che ci permette di memorizzare i nodi già visitati.

```
function GRAPH-SEARCH(problem) returns solution or failure

close ← un insieme vuoto
fringe ← INSERT(MAKE-NODE( INITIAL-STATE[problem]))
loop do
    If EMPTY?(fringe) then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST?(problem, STATE[node]) then return SOLUTION(node)
    if STATE[node] NOT in close then
        aggiungi STATE[node] in close
        child_list ← EXPAND(node, OPERATORS(problem))
        for child_node in child_list do
            if STATE[child_node] NOT in close then
                fringe ← QUEUING-FN(fringe, child_node)
        end
    end
end
```

Figure 3: Graph Search

3 Ricerca Euristica

3.1 Introduzione

La parola euristica significa, che aiuta nella scoperta. Infatti nell'algoritmo GENERAL-SEARCH si può usare conoscenza euristica per definire la QUEUING-FUNCTION ovvero ciò che determina la scelta del nodo da espandere. La conoscenza sul problema è rappresentata da una **funzione di valutazione** che si applica ai nodi dell'albero di ricerca, in particolare $f(\text{nodo}) =$ stima della desiderabilità di espandere il nodo; misura di quanto il nodo è promettente. Normalmente f è una stima del costo della soluzione, quindi viene considerato migliore il nodo n con $f(n)$ minore.

3.2 Best-First Search

La QUEUING-FN ordina i nodi dal migliore al peggiore secondo la funzione di valutazione. Il nodo scelto per l'espansione è quello la cui valutazione è migliore, cioè il nodo apparentemente più promettente.

```

function BEST-FIRST-SEARCH(problem, EVAL-FN) returns a solution or
failure

QUEUING-FN ← a function that orders nodes by EVAL-FN
fringe ← MAKE-QUEUE(MAKE-NODE(INITIAL-STATE[problem]))
loop do
  if EMPTY?(fringe) then return failure
  node ← REMOVE-FRONT(fringe)
  if GOAL-TEST(problem, STATE[node]) then return SOLUTION(node)
  fringe ← QUEUING-FN(fringe, EXPAND(node, OPERATORS(problem)))
end

```

Figure 4: Best-First Search

$\boxed{s, s_0, s_1, s_2, \dots}$: stati del problema
 $\boxed{s_0}$: stato iniziale
 $\boxed{k^*(s_i, s_j)}$: costo del cammino a costo minimo da s_i a s_j (se esiste)
 $\boxed{g^*(s_i)}$ = $k^*(s_0, s_i)$: costo del cammino a costo minimo dallo stato iniziale a s_i
 $\boxed{h^*(s_i)}$: costo effettivo di un cammino a costo minimo da s_i a uno stato obiettivo
 $\boxed{f^*(s_i)}$ = $g^*(s_i) + h^*(s_i)$: costo minimo di una soluzione vincolata a passare per s_i

Figure 5: Notazioni (1)

Notazioni che riguardano l'albero di ricerca:

$\boxed{n, n_1, n_2}$: nodi dell'albero di ricerca

Notazioni che riguardano l'albero di ricerca e costi effettivi:

$\boxed{g^*(n)}$ = $g^*(STATE(n))$

$\boxed{h^*(n)}$ = $h^*(STATE(n))$

Notazioni che riguardano l'albero di ricerca e la stima di costi:

$\boxed{g(n)}$: costo del cammino dallo stato della radice dell'albero allo stato di n
 $g(n) \geq g^*(n)$

$\boxed{h(n)}$: **stima di** $h^*(n)$ = stima del costo del cammino a costo minimo dallo stato di n a uno stato obiettivo

h : **FUNZIONE EURISTICA**

Figure 6: Notazioni (2)

3.3 Greedy Best-First Search

Ha una **Funzione di valutazione** $f = h$ che espande il nodo ritenuto più vicino a un obiettivo. Se $h(n) = 0$ vuol dire che lo stato n è un obiettivo. La ricerca golosa minimizza il costo stimato per raggiungere l'obiettivo.

3.3.1 Esempio (Ricerca Itinerario)

L'obiettivo è quello di raggiungere Bucarest a partire da Arad cercando di minimizzare il percorso.

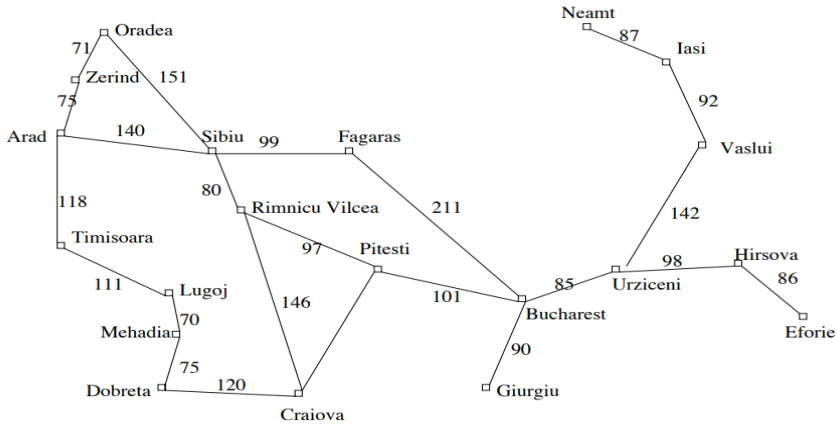


Figure 7: Mappa della Romania

SLD da Bucharest			
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Dobreta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 8: SLD da Bucarest

$h=h_{SLD}(n)$ = distanza in linea d'aria (Straight Line Distance) dallo stato n all'obiettivo

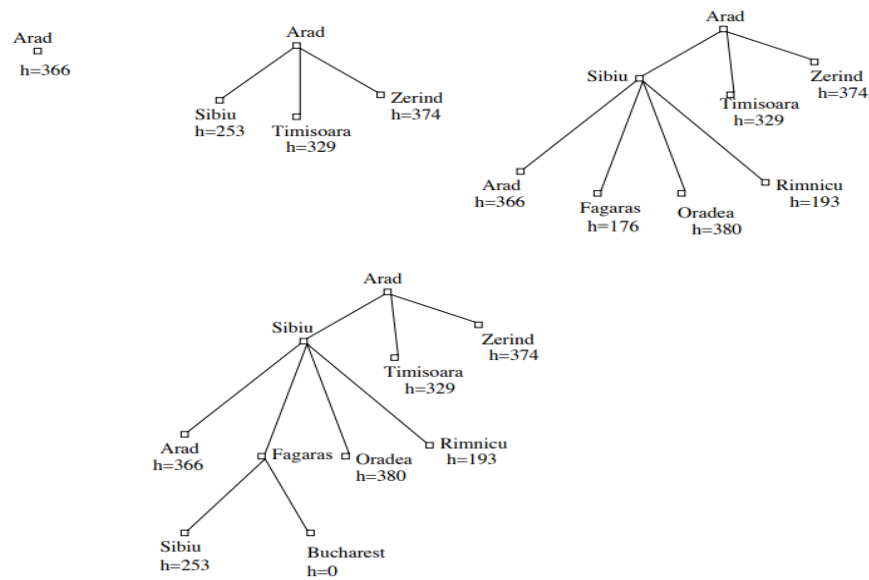


Figure 9: Ricerca golosa con funzione euristica $h=h_{SLD}$

Il problema è che questo tipo di ricerca non ci porta alla soluzione ottima. Quindi **non è completa**, infatti potremmo seguire un cammino infinito senza trovare mai la soluzione e **non è ottimale**.

3.3.2 Greedy Best-First Search codice Python

```
def Greedy_Best_First():
    # crea la frontiera
    fringe = Fringe()

    # crea lo stato iniziale
    initialState = State()

    # crea la radice
    euristica = h[initialState.name]
    root = Node(initialState, None, euristica)    # il nodo padre della radice è None

    # aggiungi alla fringe
    elemento = Elem(euristica, root)
    fringe.add(elemento)

    while not fringe.empty_fringe():             # se la fringe non è vuota ...
        elem_estratto = fringe.estrazione()       # estrazione dell'elemento in testa alla fringe
        currentNode = elem_estratto.node          # nodo estratto

        print("-- dequeue --", currentNode.state.name)

        if currentNode.state.checkGoalState():    # se lo stato del nodo estratto è lo stato obiettivo ...
            print("Stato obiettivo raggiunto")
            print("-----")
            print("Soluzione:")
            currentNode.printPath()                # stampa il percorso trovato e termina l'elaborazione
            break
```

```

else:
    # espandi il nodo estratto ottenendo i suoi nodi figli
    childStates = currentNode.state.successorFunction()
    for childState in childStates:
        euristica = h[State(childState).name]
        childNode = Node(State(childState), currentNode, euristica)

        # aggiungi il nodo figlio alla lista dei figli del nodo corrente
        currentNode.addChild(childNode)

        # aggiungi il nodo figlio alla fringe
        elemento = Elem(childNode.heuristic, childNode)
        fringe.add(elemento)
#
fringe.stampa()

```

3.4 L'Algoritmo A*

Questo algoritmo ha come **Funzione di valutazione**: $f(n) = g(n) + h(n)$ ovvero la stima del costo del cammino a costo minimo dallo stato iniziale a un obiettivo, vincolato a passare per lo stato n.

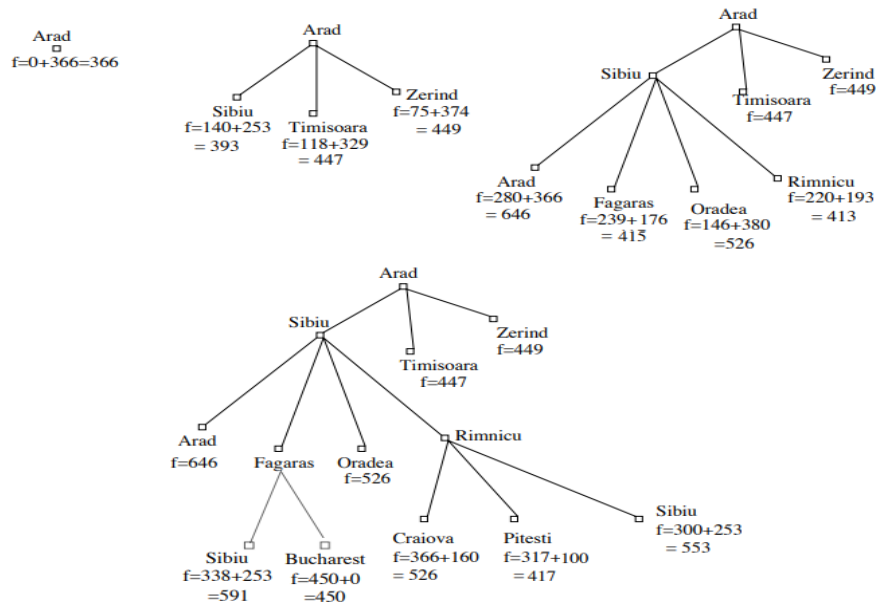


Figure 10: Visita A* (1)

$g(n)$ rappresenta il "costo accumulato" dal nodo iniziale fino al nodo n, mentre $h(n)$ è una stima del costo minimo per raggiungere l'obiettivo partendo da n. Volendo fare un esempio: osserviamo l'espansione del nodo Sibiu; $g(\text{Sibiu})=140$: rappresenta il costo del percorso dal nodo iniziale "Arad" fino a "Sibiu". $h(\text{Sibiu})=253$: rappresenta la stima del costo rimanente da "Sibiu" a "Bucharest" (quella che trovi nella tabella sopracitata).

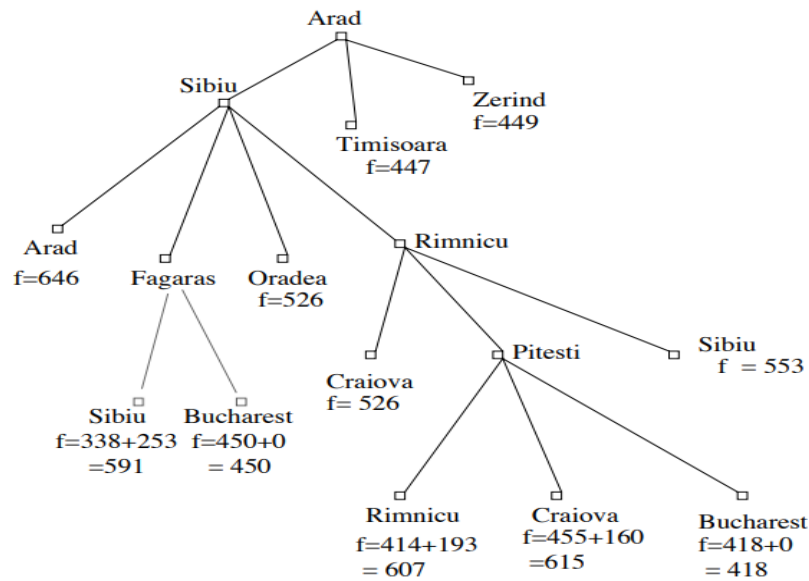


Figure 11: Visita A* (2)

3.5 Teorema di Completezza e Ottimalità per A*

Un algoritmo di ricerca si dice completo se è garantito che troverà una soluzione (se esiste). A* è completo se:

- Ogni nodo del grafo ha un numero finito di successori.
- Tutti gli archi hanno costi maggiori di una quantità positiva δ

**Se h è un'euristica ammissibile,
cioè se**

$$\forall n \quad h(n) \leq h^*(n)$$

(h "ottimista")

allora A* è completo e ottimale

Se due versioni di A*, che possiamo chiamare A_1^* e A_2^* , differiscono soltanto per il fatto che $h_1 < h_2$ per tutti i nodi non obiettivo, si dice che A_2^* è più informato di A_1^* .

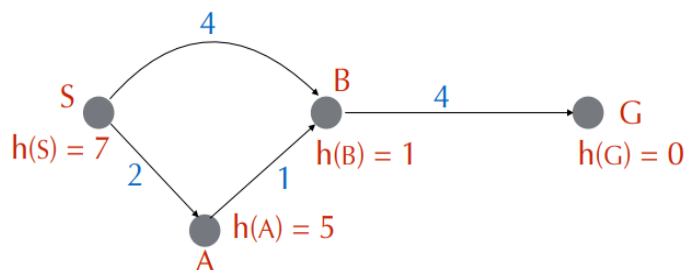
Se A_2^* è più informato di A_1^* allora, al termine delle loro ricerche su qualsiasi grafo che abbia un percorso da n_0 ad un nodo obiettivo, ogni nodo espanso da A_2^* sarà stato espanso anche da A_1^* .

Figure 12: Influenza di H sull'efficienza

Da questo teorema segue che A_1^* espande almeno tanti nodi quanti ne espande A_2^* .

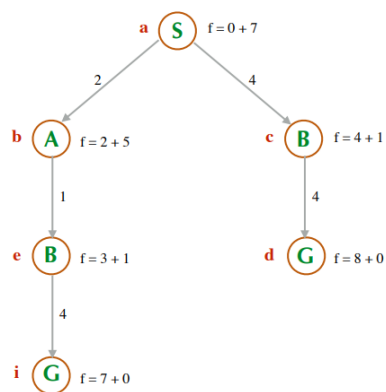
3.6 Esercizio per A* (Esercizio d'esame)

Dato il seguente spazio degli stati, con i valori della funzione euristica **h** indicati in figura:



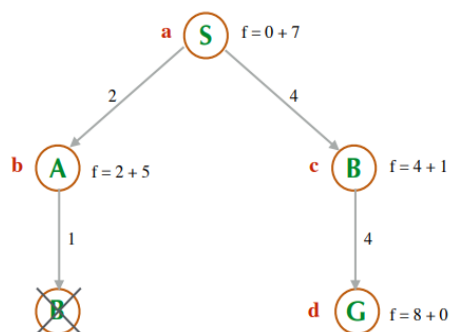
eseguire l'algoritmo **A*** sia in modalità **TREE-SEARCH** sia in modalità **GRAPH-SEARCH** e riportare le soluzioni trovate.

Vi è una differenza sostanziale nel trattamento dei nodi da parte delle due modalità:



Soluzione: **S-A-B-G**
Costo: **7**

Figure 13: Tree Search



Soluzione: **S-B-G**
Costo: **8**

Figure 14: Graph Search

Graph Search: tiene traccia dei nodi visitati (usando una struttura come una lista o un set) per evitare di riesplorare nodi già esplorati, risparmiando tempo e riducendo i duplicati.

Tree Search: non ha un controllo sui nodi già espansi. Quindi, se un nodo è raggiungibile da più percorsi, potrebbe essere riesplorato ogni volta che viene trovato tramite un percorso differente.

3.6.1 Algoritmo A* Codice Python (versione Tree Search)

```
def A_Star():

    # crea la frontiera (priority queue)
    fringe = Fringe()

    # crea il root node
    initialState = State()
    euristica = h[initialState.name]
    root = Node(initialState, None, euristica, 0) # il nodo padre di root è None

    # aggiungi alla fringe
    elemento = Elem(euristica, root)
    fringe.add(elemento)

    # esegui se la frontiera non è vuota
    while not fringe.empty_fringe():                                # se la fringe non è vuota ...

        elem_estratto = fringe.estrazione()                        # estrazione dell'elemento in testa alla fringe
        currentNode = elem_estratto.node                          # nodo estratto

        print("-- dequeue --", currentNode.state.name)

        # verifica se questo è lo stato obiettivo
        if currentNode.state.checkGoalState():
            print("Stato obiettivo raggiunto")
            # stampa il percorso trovato
            print("-----")
            print("Soluzione:")
            currentNode.printPath()
            break
```

```
        # ottieni i nodi figli del nodo estratto dalla frontiera
        childStates = currentNode.state.successorFunction()

        for (childState, distance) in childStates:
            g = currentNode.partial_path + distance
            euristica = h[childState]
            f = g + euristica
            childNode = Node(State(childState), currentNode, f, g)

        #          currentNode.addChild(childNode)                # aggiungi alla lista dei figli

        # aggiungi il nodo figlio alla fringe
        elemento = Elem(childNode.heuristic, childNode)
        fringe.add(elemento)

    #          fringe.stampa()
```

3.6.2 Algoritmo A* Codice Python (versione Graph Search)

```
def A_Star():

    # crea la frontiera (priority queue)
    fringe = Fringe()

    # crea la lista dei nodi visitati
    close = []

    # crea il root node
    initialState = State()
    euristica = h[initialState.name]
    root = Node(initialState, None, euristica, 0) # il nodo padre di root è None

    # inserisci root nella frontiera
    elemento = Elem(euristica, root)
    fringe.add(elemento)

    # esegui se la frontiera non è vuota
    while not fringe.empty_fringe(): # se la fringe non è vuota ...

        elem_estratto = fringe.estrazione() # estrazione dell'elemento in testa alla fringe
        currentNode = elem_estratto.node # nodo estratto

        print("-- dequeue --", currentNode.state.name)

        # verifica se questo è lo stato obiettivo
        if currentNode.state.checkGoalState():
            print("Stato obiettivo raggiunto")
            # stampa il percorso trovato
            print("-----")
            print("Soluzione:")
            currentNode.printPath()
            break
```

```
    # verifica se lo stato del nodo estratto sta in close
    if currentNode.state.name not in close:

        # aggiungiamo lo stato di questo nodo alla lista degli stati visitati
        close.append(currentNode.state.name)

        # ottieni i nodi figli del nodo estratto dalla frontiera
        childStates = currentNode.state.successorFunction()

        for (childState, distance) in childStates:
            g = currentNode.partial_path + distance
            euristica = h[childState]
            f = g + euristica
            childNode = Node(State(childState), currentNode, f, g)

            # verifica se lo stato del nodo figlio sta in close
            if childNode.state.name not in close:

                # aggiungi il nodo figlio alla fringe
                elemento = Elem(childNode.heuristic, childNode)
                fringe.add(elemento)

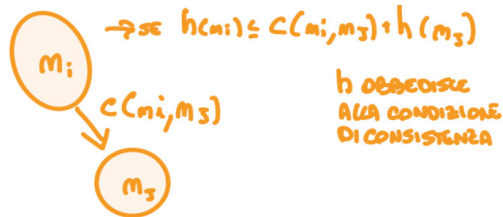
        fringe.stampa()
```

3.7 La condizione di consistenza

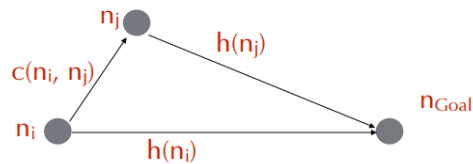
Consideriamo una coppia di nodi in cui n_j è un successore di n_i . Diciamo che h obbedisce alla condizione di consistenza se per tutte le coppie di nodi di questo tipo nel grafo di ricerca:

$$h(n_j) \leq c(n_i, n_j) + h(n_i)$$

Dove $c(n_i, n_j)$ è il costo dell'arco da n_i a n_j .



La condizione di consistenza può anche essere considerata come un tipo di disuguaglianza triangolare:



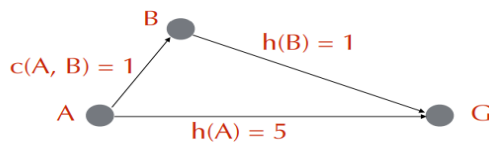
$$h(n_i) \leq c(n_i, n_j) + h(n_j)$$

Dati n_i e n_j , con n_j successore di n_i :

$$f(n_j) \geq f(n_i)$$

3.7.1 Teorema

Se la condizione di consistenza su h è soddisfatta, allora quando A^* espande un nodo n , esso ha già trovato un percorso ottimo per n . Infatti nel Graph Search di prima:



$$h(A) \leq c(A, B) + h(B) ?$$

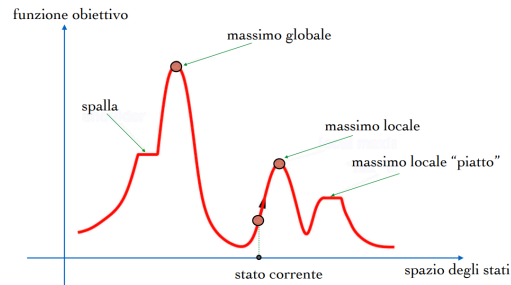
$$5 \leq 1 + 1 ?$$

NO!

4 Local Search

4.1 Introduzione

Ci sono dei problemi in cui lo stato obiettivo contiene tutte le informazioni rilevanti per la soluzione. In tali casi il cammino verso l'obiettivo è irrilevante. Gli algoritmi di ricerca locale si muovono sulla superficie cercando i picchi più alti



4.2 Hill Climbing

Il termine **Hill Climbing** che letteralmente significa "scalatore di colline", deriva dal fatto che segue sempre le salite più rapide. Si tratta di un semplice ciclo che si muove continuamente verso l'alto, cioè nella direzione dei valori crescenti, e termina quando raggiunge un picco che non ha vicini di valore più alto.

4.2.1 Steepest Ascent Hill Climbing

Questo algoritmo segue le salite più ripide e può incappare in massimi locali.

```
function HILL-CLIMBING(problem) returns uno stato che è un
    massimo locale
inputs: problem, un problema
local variables: current, un nodo
                 next, un nodo

current ← MAKE-NODE( INITIAL-STATE( problem ) )
loop do
    next ← successore di current di valore più alto
    if VALUE( next ) < VALUE( current ) then return STATE( current )
    current ← next
end
```

Figure 15: Steepest Ascent

4.2.2 Random Restart Hill Climbing

L'algoritmo è in un Hill-Climbing con riavvio casuale. Esso conduce una serie di ricerche Hill Climbing partendo da stati iniziali generati casualmente. L'algoritmo è completo, con probabilità tendente a 1, per la banale ragione che prima o poi dovrà generare, come stato iniziale, proprio un obiettivo.

```
function RANDOM-RESTART-HILL-CLIMBING(problem) returns uno stato  
soluzione  
inputs: problem, un problema  
local variables: current, next, best: nodi; local: booleana  
t ← 0  
Inizializza best  
repeat  
    local ← false  
    seleziona un punto iniziale current random  
    repeat ← RICHIAMO HILL CLIMBING  
        next ← successore di current con VALUE più alto  
        if VALUE(next) > VALUE(current)  
            then current ← next  
            else local ← true  
    until local ← HILL CLIMBING CI HA RESTITUITO UN PICCO  
    t ← t + 1  
    if VALUE(current) > VALUE(best) then best ← current  
until t = MAX  
return STATE(best)  
end
```

Figure 16: Random Restart

Il ciclo interno della procedura restituisce sempre un ottimo locale (con il numero delle scelte all'infinito trovo l'ottimo). La procedura sfugge dagli ottimi locali solo avviando una nuova ricerca partendo da un nuovo punto scelto casualmente.

4.2.3 Stochastic Hill-Climbing

L'algoritmo Stochastic Hill-Climbing si ottiene modificando la procedura precedente: anzichè valutare tutti i punti vicini a current e selezionare il migliore, selezioniamo casualmente solo un punto next (questo meccanismo gli permette di sfuggire dagli ottimi locali dato che potrebbe essere scelto un next peggiore). Accettiamo next con una probabilità che dipende dalla differenza di valutazione tra i due punti:

$$\Delta E = VALUE(current) - VALUE(next)$$

```
function STOCHASTIC-HILL-CLIMBING(problem) returns uno stato soluzione  
inputs: problem, un problema  
local variables: current, next, best: nodi  
t ← 0  
seleziona un punto iniziale current random  
Inizializza best  
repeat  
    next ← seleziona successore di current random  
    seleziona next come nodo corrente con probabilità  $p = 1/(1 + e^{\Delta E/T})$   
    se selezionato next come nuovo current:  
        if VALUE(current) > VALUE(best) then best ← current  
    t ← t + 1  
until t = MAX  
return STATE(best)  
end
```

Figure 17: Stochastic Hill-Climbing

Il nuovo punto accettato viene selezionato con probabilità p . Al crescere di T diventa sempre meno importante la differenza delle valutazioni dei due punti e la probabilità di accettazione diminuisce. Al variare di next invece quello che succede è che: andare a un next peggiore è possibile ma con probabilità molto bassa, se ha lo stesso valore di current la probabilità è del 50% mentre al crescere di next la probabilità di accettazione aumenta. Il problema risolto nell'algoritmo è relativo al passaggio dai valori di $\frac{\Delta E}{T}$ a quelli delle probabilità. $\frac{\Delta E}{T}$ ha un range che va da $-\infty$ a $+\infty$ la probabilità invece va da 0 a 1. L'espressione usata nell'algoritmo per la probabilità è in buona sostanza una "link function" che realizza un mapping tra i due intervalli:

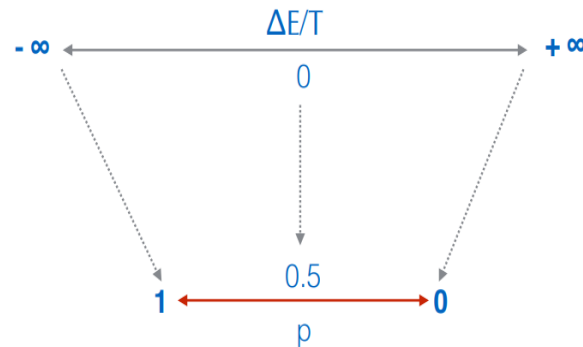


Figure 18: Link Function

4.2.4 Hill Climbing codice Python

La versione di Hill Climbing che andremo a implementare si chiama **First-Choice Hill Climbing**. Esso, dato uno stato corrente, sceglie casualmente un successore, e passa al nuovo stato se la valutazione di tale successore è migliore di quella dello stato corrente.

```
def ricerca_stringa(stringa_iniziale):
    current_state = stringa_iniziale
    current_eval = evaluate(current_state)
    while True:
        print('Punteggio attuale:', current_eval, 'Stringa:', "".join(current_state))

        if current_eval == 0:
            break

        next_state = list(current_state)      # scelta random
        tweak(next_state)                     # del successore

        next_state_eval = evaluate(next_state) # valutazione del successore scelto

        if next_state_eval < current_eval:    # se next è migliore ...
            current_state = next_state        # ... esso diventa
            current_eval = next_state_eval    # il nuovo nodo corrente
```

4.3 Simulated Annealing

L'algoritmo di Simulated Annealing prende il nome dall'analogia con il processo di ricottura in metallurgia che consiste nel ridurre gradualmente la temperatura di un materiale in modo da permettere alla sua struttura cristallina di raggiungere uno stato ad energia minima. È un algoritmo con miglioramenti iterativi che consente di sfuggire da ottimi locali durante la fase di search.

La differenza principale tra lo **stochastic hill-climber** e il **simulated annealing** è che quest'ultimo modifica il parametro **T** durante l'esecuzione. Il simulated annealing comincia con valori alti di **T** comportandosi in modo simile ad una pura **random search**. Poi decrementa gradualmente il valore di **T**. Verso la fine dell'esecuzione i valori di **T** sono molto piccoli e perciò l'algoritmo si comporta in modo simile ad un normale **hill-climber**. Inoltre l'algoritmo accetta sempre nuovi punti se essi hanno una valutazione migliore del punto corrente.

```
function SIMULATED-ANNEALING(problem) returns uno stato soluzione
inputs: problem, un problema
local variables: current, next: nodi;
t ← 0
Inizializza T
seleziona un punto iniziale current random
Inizializza best
repeat
  repeat
    next ← successore di current scelto random
    if VALUE(next) > VALUE(current)
      then current ← next
      if VALUE(current) > VALUE(best) then best ← current
    else if random[0, 1) <  $e^{-\Delta E/T}$  then current ← next
  until (termination-condition)
  T = g(T, t)
  t ← t + 1
until (halting-criterion)
return STATE(best)
end
```

Annotations in the original image:
- Next to $e^{-\Delta E/T}$: "DE E TAVOL 0,1 C'È IL 10% CHE CURRENT DIVENTA NEXT"
- Next to "termination-condition": "CONDIZIONE DI TERMINAZIONE"
- Next to "halting-criterion": "CONDIZIONE DI FERMATA"

Figure 19: Simulated Annealing

4.3.1 Simulated Annealing codice Python

La funzione Tweak: Viene utilizzata per scegliere random uno stato successore. **La funzione inizializza:** Serve per scegliere casualmente lo stato di partenza. **La funzione energia:** È la funzione di valutazione

```
def simulated_annealing():
    # impostazione dello stato iniziale
    current = inizializza(range(0, DIMENSIONE))
    current_energy = energia(current)

    # inizializzazione best
    best = current
    best_energy = current_energy

    temperature = TEMPERATURA_INIZIALE

    while (temperature > TEMPERATURA_FINALE and best_energy != 0):
        print("TEMPERATURA: "),
        print(temperature)
        for step in range(0, STEPS_PER_CHANGE):
            useNew = False
            next_state = tweak(current)
            next_state_energy = energia(next_state)

            if (next_state_energy < current_energy):
                useNew = True
            else:
                delta = next_state_energy - current_energy
                metropolis = math.exp(-delta/temperature)
                test = random.random()
                if (test < metropolis):
                    useNew = True

            # scelta random dello stato successore nel neighbourhood
            # valutazione dell'energia dello stato successore
            # se il successore è migliore lo accettiamo
            # ... accettiamo il nuovo stato
```

```

# se abbiamo deciso di accettare il nuovo stato:
if (useNew):
    # impostalo come stato ed energia correnti
    current = next_state
    current_energy = next_state_energy

    # se è anche il migliore segna il record
    if (current_energy < best_energy):
        best = current
        best_energy = current_energy

print("best_energy="),
print(best_energy)

# diminuisci la temperatura, senza mai arrivare a zero
temperature = temperature * ALFA

return(best)

```

5 Strutture Dati in Python

5.1 Liste Concatenate

5.1.1 Liste singolarmente concatenate

Una lista contiene dei nodi che hanno dei dati e un puntatore al nodo successivo.

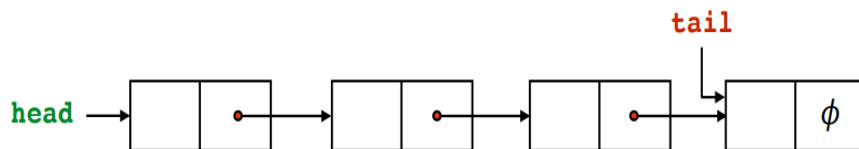


Figure 20: Rappresentazione grafica lista

Definizione della classe nodo:

```

class Node:
    data = ''
    next = None

    def __init__(self, data, next):
        self.data = data
        self.next = next

```

Figure 21: Classe Nodo

Definizione della classe lista: (inizializzata a lista vuota)

```
class SinglyLinkedList:
#     __head = None
#     __tail = None

    def __init__(self):
        self.__head = None
        self.__tail = None
```

Figure 22: Classe lista

Per **scansionare la lista** bisogna partire impostando un puntatore alla testa

```
1 p = self.__head
```

e si procede fino a raggiungere l'ultimo elemento della lista

```
1 p = p.next
```

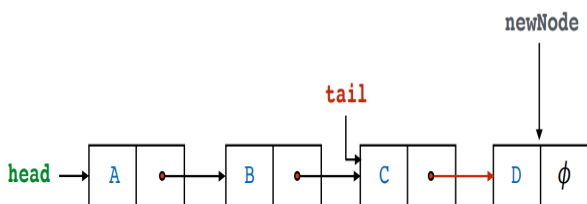
ovvero quando `p.next=None`

Definizione della funzione stampa_lista (`end=' '` serve a non andare a capo alla fine della stampa, ma a lasciare uno spazio)

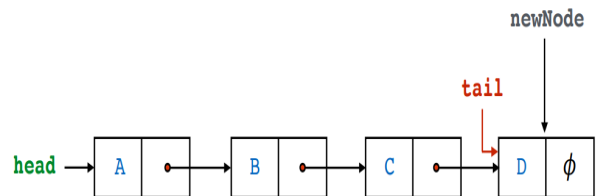
```
def stampa_lista(node):
    p = node
    while p != None:
        data = p.data
        print(data, '->', end = ' ')
        p = p.next
    print('Fine Lista')
```

Figure 23: Funzione stampa_lista

Inserimento in coda (APPEND):



Istruzione: `self.__tail.next = newNode`



Istruzione: `self.__tail = newNode`

```

1 self.__tail=newNode
2 self.__head=newNode

```

In caso la lista fosse vuota si impostano i puntatori di testa e coda al nuovo nodo.

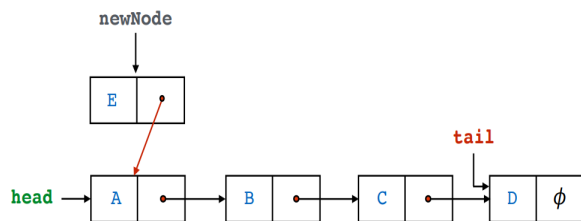
```

def append(self, newNode):
    if self.__head == None:
        self.__head = newNode
        self.__tail = newNode
        newNode.next = None
    else:
        self.__tail.next = newNode
        self.__tail = newNode
        newNode.next = None

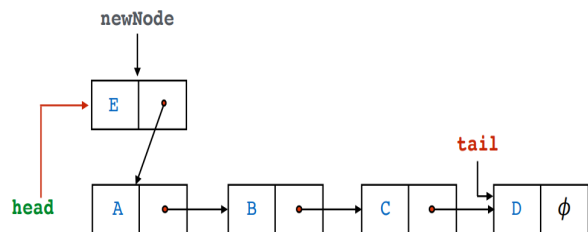
```

Figure 24: Inserimento in coda

Inserimento in testa



Istruzione: `newNode.next = self.__head`



Istruzione: `self.__head = newNode`

Se la lista fosse vuota si procederebbe come per l'append.

```

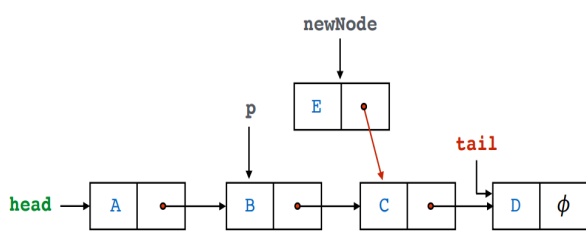
def insert_head(self, newNode):
    if self.__head == None:
        self.__head = newNode
        self.__tail = newNode
        newNode.next = None
    else:
        newNode.next = self.__head
        self.__head = newNode

```

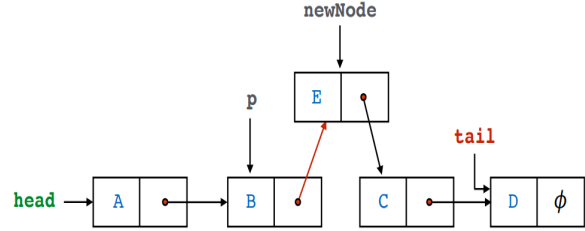
Figure 25: Inserimento in testa

Inserimento di un elemento in posizione intermedia

Supponiamo di creare un nuovo nodo e di volerlo mettere in posizione $i=2$ (i parte da 0). Si procede fino a raggiungere la posizione precedente quella dell'inserimento (ovviamente il puntatore parte da `self.__head`).



Istruzione: `newNode.next = p.next`



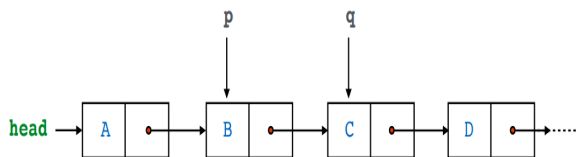
Istruzione: `p.next = newNode`

```
def insert_position(self, insertPosition, newNode):  
    p = self.__head  
    i = 0  
    # Spostiamo il puntatore p nella posizione di inserimento  
    while p != None and i < insertPosition - 1:  
        p = p.next  
        i = i + 1  
    newNode.next = p.next  
    p.next = newNode
```

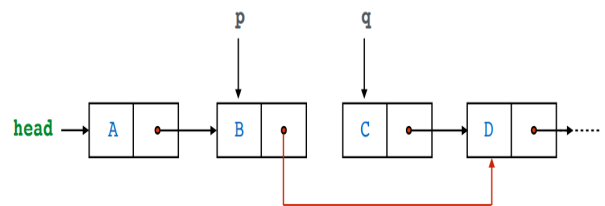
Figure 26: Inserimento di un elemento in posizione intermedia

Cancellazione dell'elemento i-esimo

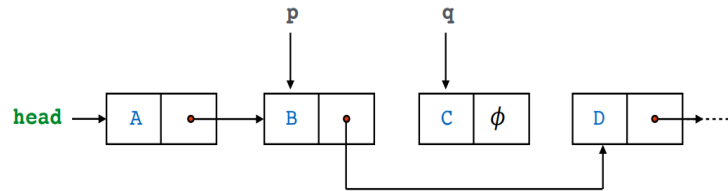
Come per l'inserimento si procede fino a raggiungere la posizione precedente a quella della cancellazione.



Istruzione: `q = p.next`



Istruzione: `p.next = q.next`



Istruzione: `q.next = None`

```

def delete(self, deletePosition):
    p = self.__head
    if p != None:
        if deletePosition == 0:
            self.__head = p.next
            p.next = None
        else:
            i = 0
            # Spostiamo il puntatore sul nodo immediatamente precedente
            # a quello che vogliamo rimuovere
            while p.next != None and i < (deletePosition - 1):
                p = p.next
                i += 1
            q = p.next
            p.next = q.next
            q.next = None
    else:
        print('Lista vuota')
  
```

Figure 27: Cancellazione dell'elemento i-esimo

5.2 Liste Ordinate

Una lista ordinata è una lista i cui elementi sono ordinati secondo una certa chiave. Ogni operazione di inserimento di un nuovo elemento deve dunque rispettare l'ordinamento suddetto.

Definizione della classe Nodo

```

class Node:
    data = None
    next = None

    def __init__(self, data):
        self.data = data
  
```

Figure 28: Definizione della classe nodo

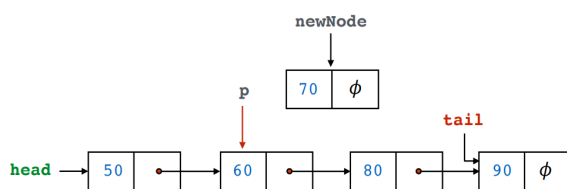
Definizione della classe lista

```
class SortedLink:
#     __head = None
#     __tail = None

    def __init__(self):
        self.__head = None
        self.__tail = None
```

Figure 29: Definizione della classe nodo

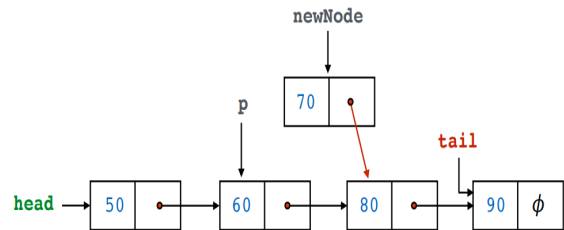
Inserimento in posizione intermedia



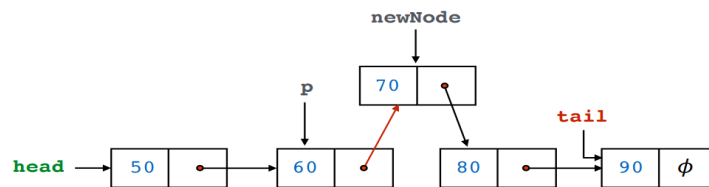
Istruzione: `p = p.next`

Si procede fino a quando vale la condizione:

`newNode.data > p.next.data`



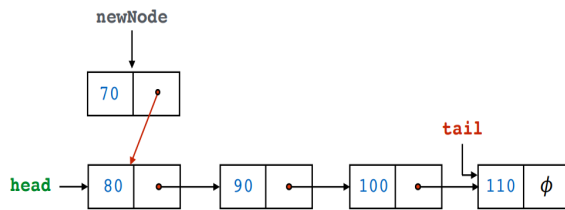
Istruzione: `newNode.next = p.next`



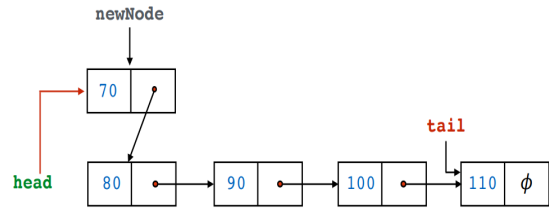
Istruzione: `p.next = newNode`

Figure 30: Inserimento in posizione intermedia

ovviamente c'è da considerare anche il caso in cui il valore del nodo si trovi prima dell'inizio della lista.



Istruzione: `newNode.next = self.__head`



Istruzione: `self.__head = newNode`

In caso di lista vuota il procedimento di inserimento è quello di una lista singolarmente concatenata.

```
def add(self, newNode):
    p = self.__head
    if (self.__head == None):
        # se la lista è vuota ...
        # inserisci l'elemento
        self.__head = newNode
        self.__tail = self.__head
        newNode.next = None

    elif (newNode.data > self.__tail.data):
        # se il valore è maggiore dell'ultimo ...
        # fai una append
        self.__tail.next = newNode
        self.__tail = newNode
        newNode.next = None

    elif (newNode.data < self.__head.data):
        # se è minore del primo ...
        # inserisci in testa
        newNode.next = self.__head
        self.__head = newNode

    else:
        while (p.next != None and (newNode.data > p.next.data)):
            # scandisci la lista
            # fino al punto di inserimento
            p = p.next
        newNode.next = p.next
        p.next = newNode
```

Figure 31: Inserimento di un elemento

5.3 Le pile

Una pila è un tipo di dato astratto, basato sul modello dei dati delle liste, in cui tutte le operazioni vengono effettuate a un estremo della lista, chiamato la **testa (top)**. La politica di gestione è la **Last-In-First-Out (LIFO)**.

Definizione della classe nodo:

```
class Node:
    data = ''
    next = None

    def __init__(self, data):
        self.data = data
```

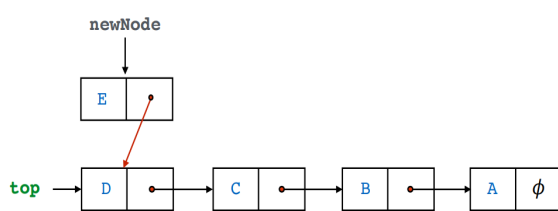
Figure 32: Definizione della classe nodo

Definizione della classe Stack (pila)

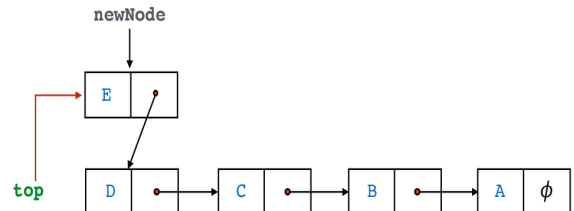
```
class Stack:
    __top = None
    __size = 0
```

Figure 33: Definizione della classe Stack

Inserimento di un elemento (PUSH)



Istruzione: `newNode.next = self.__top`

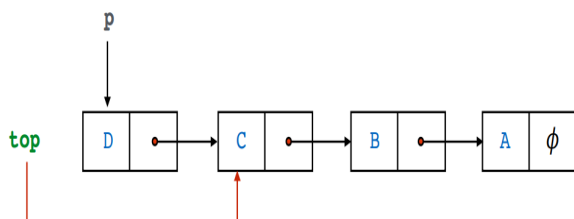


Istruzione: `self.__top = newNode`

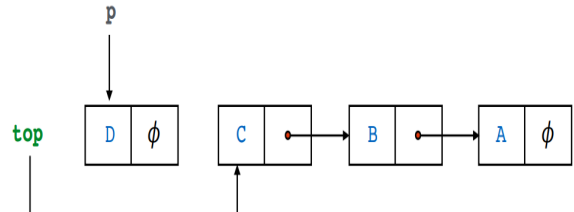
```
def push(self, element):
    if self.__top == None:
        self.__top = Node(element)
    else:
        newNode = Node(element)
        newNode.next = self.__top
        self.__top = newNode
        self.__size += 1
```

Figure 34: Inserimento di un elemento (PUSH)

Estrazione di un elemento (POP)



Istruzione: `self.__top = self.__top.next`



Istruzione: `p.next = None`

```
def pop(self):
    if self.__top == None:
        return None
    p = self.__top
    self.__top = self.__top.next
    p.next = None
    self.__size -= 1
    return p
```

Figure 35: Estrazione di un elemento (POP)

5.4 Le code

Un altro importante tipo di dato astratto basato sul modello dei dati lista è la coda, una forma ristretta di lista in cui gli elementi vengono inseriti a un estremo, il retro (**tail**), ed estratti all'altro estremo, la fronte o testa (**head**). La politica di gestione è dunque la **First-In-First-Out (FIFO)**.

Definizione della classe nodo

```
class Node:
    data = ''
    next = None

    def __init__(self, data):
        self.data = data
```

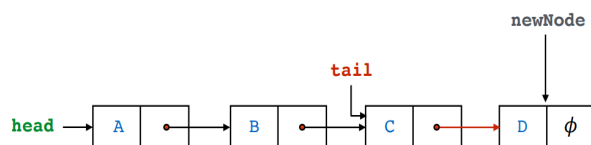
Figure 36: Definizione della classe nodo

Definizione della classe Queue

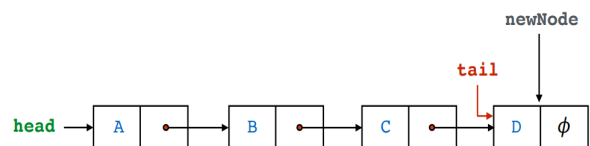
```
class Queue:
    __head = None
    __tail = None
    __size = 0
```

Figure 37: Definizione della classe Queue

Inserimento di un elemento (Enqueue)



Istruzione: `self.__tail.next = newNode`



Istruzione: `self.__tail = newNode`

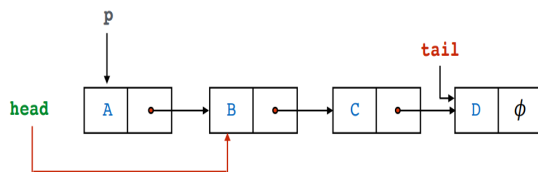
```

def enqueue(self, element):
    newNode = Node(element)
    if self.__head == None:
        self.__head = newNode
        self.__tail = self.__head
    else:
        self.__tail.next = newNode
        self.__tail = newNode
    self.__size += 1

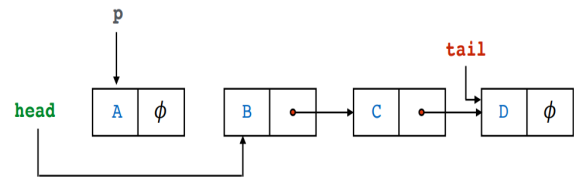
```

Figure 38: Inserimento di un elemento (Enqueue)

Estrazione di un elemento (Dequeue)



Istruzione: `self.__head = self.__head.next`



Istruzione: `p.next = None`

```

def dequeue(self):
    p = self.__head
    if p == None:
        return None
    self.__head = self.__head.next
    p.next = None
    self.__size -= 1
    return p

```

Figure 39: Estrazione di un elemento (Dequeue)

5.5 Introduzioni agli algoritmi Python

5.5.1 Greedy Best-First Search

L'algoritmo Greedy Best-First Search anche detto ricerca golosa, è un algoritmo che fa parte della ricerca euristica, brevemente la ricerca euristica utilizza una Queuing-FN (implementata nella frontiera) la quale ordina i nodi dal migliore al peggiore secondo la funzione di valutazione. Ovviamente il nodo scelto per l'espansione è quello la cui valutazione è "migliore". L'algoritmo Greedy ha **funzione di valutazione** $f = h$ (dove h è l'euristica, la quale ci aiuta a determinare la scelta sul nodo da espandere), infatti espande il nodo ritenuto più vicino a un obiettivo. Se $h(n) = 0$ vuol dire che lo stato n è un obiettivo. La ricerca golosa minimizza il costo stimato per raggiungere l'obiettivo. Nel problema della ricerca di un itinerario vogliamo raggiungere da una città di partenza, l'obiettivo a partire da Arad cercando di minimizzare il percorso. Questo algoritmo non è ne ottimale ne completo. Nell'implementazione Python viene prima definito un dizionario che rappresenta il grafo degli stati, e un dizionario per la definizione dell'euristica.

5.5.2 L'algoritmo A*

L'algoritmo A*, è un algoritmo che fa parte della ricerca euristica, brevemente la ricerca euristica utilizza una Queuing-FN (implementata nella frontiera) la quale ordina i nodi dal migliore al peggiore secondo la funzione di valutazione. Ovviamente il nodo scelto per l'espansione è quello la cui valutazione è "migliore". Questo algoritmo ha come **funzione di valutazione** $f(n) = g(n) + h(n)$ ovvero la stima del costo del cammino a costo minimo dallo stato iniziale a un obiettivo, vincolato a passare per lo stato n . $g(n)$ rappresenta il "costo accumulato" dal nodo iniziale fino al nodo n , mentre $h(n)$ (l'euristica) in questo caso, è una stima del costo minimo per raggiungere l'obiettivo partendo da n . Nel problema della ricerca di un itinerario vogliamo raggiungere da una città di partenza, l'obiettivo a partire da Arad cercando di minimizzare il percorso.

5.5.3 Hill Climbing

L'algoritmo Hill Climbing fa parte della categoria della ricerca locale. Ci sono problemi in cui lo stato obiettivo contiene tutte le informazioni rilevanti per la soluzione, infatti in questi casi il cammino verso l'obiettivo è irrilevante. Gli algoritmi di ricerca locale si muovono sulla superficie cercando i picchi più alti. Infatti il termine Hill Climbing deriva proprio da questo, infatti l'algoritmo segue sempre le salite più rapide. La versione che implementiamo in Python si chiama **First-Choice Hill Climbing**, esso dato lo stato corrente, sceglie casualmente un successore, e passa al nuovo stato se la valutazione di tale successore è migliore di quella dello stato corrente.

5.5.4 Simulated Annealing

L'algoritmo Hill Climbing fa parte della categoria della ricerca locale. Ci sono problemi in cui lo stato obiettivo contiene tutte le informazioni rilevanti per la soluzione, infatti in questi casi il cammino verso l'obiettivo è irrilevante. Gli algoritmi di ricerca locale si muovono sulla superficie cercando i picchi più alti. L'algoritmo di Simulated Annealing prende il nome dall'analogia con il processo di ricottura in metallurgia che consiste nel ridurre gradualmente la temperatura di un materiale in modo da permettere alla sua struttura cristallina di raggiungere uno stato ad energia minima. Il Simulated Annealing comincia con valori alti di T comportandosi in modo simile ad una pura random search. Poi decrementa gradualmente il valore di T . Verso la fine dell'esecuzione i valori di T sono molto piccoli e perciò l'algoritmo si comporta in modo simile ad un normale hill-climber. Inoltre l'algoritmo accetta sempre nuovi punti se essi hanno una valutazione migliore del punto corrente. **La funzione tweak** viene utilizzata per scegliere random uno stato successore. **La funzione inizializza** serve per scegliere casualmente lo stato di partenza. **La funzione energia** è la funzione di valutazione.